

ה א ו נ י ב ר ס י ט ה ה פ ת ו ח ה

20905

שפות תכנות

חוברת הקורס- אביב 2026ב

כתב: דני כלפון

מרץ 2026 - סמסטר אביב – תשפ"ו

פנימי – לא להפצה.

© כל הזכויות שמורות לאוניברסיטה הפתוחה.

תוכן העניינים

א	אל הסטודנט
ב	1. לוח זמנים ופעילויות
ד	2. תיאור המטלות
ד	2.1 מידע כללי
ד	2.2 מבנה המטלות
ה	3. התנאים לקבלת נקודות זכות
1	ממ"ן 11
3	ממ"ן 12
7	ממ"ן 13
9	ממ"ן 14
15	ממ"ן 15
21	ממ"ן 16

אל הסטודנט,

אנו מקדמים את פניך בברכה עם הצטרפותך אל הלומדים בקורס "שפות תכנות". בחוברת זו תמצא לוח הזמנים של הקורס, מטלות ותנאים לקבלת נקודות זכות בקורס.

לקורס קיים אתר באינטרנט בו תמצאו חומרי למידה נוספים, אותם מפרסם/מת מרכז/ת ההוראה. בנוסף, האתר מהווה עבורכם ערוץ תקשורת עם צוות ההוראה ועם סטודנטים אחרים בקורס. פרטים על למידה מתוקשבת ואתר הקורס, תמצאו באתר שה"ם בכתובת:

<http://telem.openu.ac.il>

מידע על שירותי ספרייה ומקורות מידע שהאוניברסיטה מעמידה לרשותכם, תמצאו באתר הספרייה באינטרנט www.openu.ac.il/Library.

שעות הייעוץ הטלפוני שלי יפורסמו סמוך לפתיחת הסמסטר באתר הקורס. אפשר ורצוי לפנות אלי בדואר אלקטרוני: dannyc@openu.ac.il, תוך ציון שם מלא, מספר תעודת זהות ומספר טלפון. פגישות חובה לתאם מראש. לצורך בירורים בנושאים אדמיניסטרטיביים יש לפנות בכתב או טלפונית למחלקת האוניברסיטה הפתוחה.

הקורס הוא תכנותי באופיו, וכולל מטלות תכנותיות החשובות להבנת החומר ותרגולו. מומלץ שתקדישו זמן ראוי ללמידת חומר הקורס, שכן זה קורס השונה באופיו מקורסים תכנותיים אחרים המוכרים לכם באו"פ. הקורס כולל פיתוח מפרשים לשפות תכנות פשוטות המדגימות עקרונות הקיימים בשפות תכנות מודרניות. השתתפות במפגשים, הקדשת זמן ראוי ללמידת החומר והגשת המטלות הם הדרך הנכונה לסיום הקורס בהצלחה.

בברכת לימוד מהנה

כלפון דני

מרכז ההוראה בקורס

לוח זמנים ופעילויות (20905 / ב2026)

שבוע לימוד	תאריכי שבוע הלימוד	יחידת הלימוד המומלצת	מפגשי ההנחיה*	תאריך אחרון למשלוח ממ"ן (למנחה)
1	13.03.2026-08.03.2026	סביבת העבודה ושפת scheme ופרק 1	מפגש ראשון	
2	20.03.2026-15.03.2026	סביבת העבודה ושפת scheme ופרק 1		
3	27.03.2026-22.03.2026	פרק 2	מפגש שני	
4	03.04.2026-29.03.2026 (ד-ו פסח)	פרק 2		ממ"ן 11 29.3.2026
5	10.04.2026-05.04.2026 (א-ד פסח)	פרק 3	מפגש שלישי	
6	17.04.2026-12.04.2026 (ב יום הזכרון לשואה)	פרק 3		
7	24.04.2026-19.04.2026 (ג יום הזיכרון, ד יום העצמאות)	פרק 3	מפגש רביעי	ממ"ן 12 19.04.2026
8	01.05.2026-26.04.2026	פרק 3		
9	08.05.2026-03.05.2026 (ה ל"ג בעומר)	פרק 3	מפגש חמישי	

* התאריכים המדויקים של המפגשים הקבוצתיים מופיעים ב"לוח מפגשים ומנחים".

לוח זמנים ופעילויות - המשך

שבוע הלימוד	תאריכי שבוע הלימוד	יחידת הלימוד המומלצת	מפגשי ההנחיה*	תאריך אחרון למשלוח הממ"ן (למנחה)
10	15.05.2026-10.05.2026	פרק 4		ממ"ן 13 10.05.2026
11	22.05.2026-17.05.2026 (ה-ו שבועות)	פרק 4	מפגש שישי	
12	29.05.2026-24.05.2026	פרק 4		ממ"ן 14 24.05.2026
13	05.06.2026-31.05.2026	פרק 7	מפגש שביעי	
14	12.06.2026-07.06.2026	פרק 7		ממ"ן 15 7.06.2026
15	19.06.2026-14.06.2026	חזרה לבחינה	מפגש שמיני	ממ"ן 16 14.06.2026

מועדי בחינות הגמר יפורסמו בנפרד

* התאריכים המדויקים של המפגשים הקבוצתיים מופיעים ב"לוח מפגשים ומנחים".

2. תיאור המטלות

לתשומת לבכם!

כדי לעודדכם להגיש לבדיקה מספר רב של מטלות הנהגנו את ההקלה שלהלן:

אם הגשתם מטלות מעל למשקל המינימלי הנדרש בקורס, **המטלות** בציון הנמוך ביותר, שציוניהן נמוכים מציון הבחינה (**עד שתי מטלות**), לא יילקחו בחשבון בעת שקלול הציון הסופי.

זאת בתנאי שמטלות אלה **אינן חלק מדרישות החובה בקורס** ושהמשקל הצבור של המטלות האחרות שהוגשו, מגיע למינימום הנדרש.

זכרו! ציון סופי מחושב רק לסטודנטים שעברו את בחינת הגמר בציון 60 ומעלה והגישו מטלות כנדרש באותו קורס.

שימו לב!

בקורס זה **חובה** להגיש את המטלות בזמן, בהתאם לתאריך ההגשה המצוין עליהן. במקרים חריגים, כאשר יש סיבה מוצדקת להגשת המטלה באיחור, יש לפנות **בכתב** בדואר אלקטרוני אל מרכז ההוראה בקורס. **את הבקשה יש להגיש מראש!** יש לצרף לבקשה אישורים רשמיים, להצדקת סיבת הבקשה.

מטלות שיוגשו באיחור ללא אישור יבדקו והציון שיוזן עבורן יהיה 0, ללא תלות בציון של הבדיקה.

שימו לב, טיפול בבקשות שנשלחות לאחר מועד ב' של הסמסטר אינו בסמכות מרכז ההוראה, ויש להפנות את האחראית על פניות סטודנטים של החטיבה למדעי המחשב.

להלן פירוט הניקוד לכל מטלה:

ניקוד	ממ"ן
5	11
5	12
5	13
5	14
5	15
5	16

3. התנאים לקבלת נקודות זכות

כדי לקבל נקודות זכות בקורס זה עליך לעמוד בדרישות הבאות:

(א) **צבירת משקל של לפחות 20 נקודות במטלות.**

(ב) ציון של לפחות 60 נקודות בבחינת הגמר.

(ג) ציון סופי בקורס של 60 נקודות לפחות.

לתשומת לבכם:

מדיניות קורס זה היא לאשר הזנת ציון אפס במטלות שלא הוגשו כנדרש בקורס. סטודנטים אשר לא הגישו את מכסת המטלות המינימאלית לעמידה בדרישות הקורס ולקבלת זכאות להיבחן, ומבקשים שמטלות חסרות יוזנו בציון אפס, יפנו למוקד הפניות והמידע

בטלפון **09-7782222** או **יעדכנו בעצמם** באתר שאילתא <http://www.openu.ac.il/sheilta>

קורסים ⇨ ציוני מטלות ובחינות ⇨ הזנת ציון 0 למטלות רשות שלא הוגשו.

יש לקחת בחשבון כי מטלות אשר יוזן להן ציון אפס ישוקללו בחישוב הציון הסופי ובכך יורידו ציון זה ולא ניתן יהיה להמירן במטלות חלופיות במועד מאוחר יותר. על כן קיימת אפשרות שסטודנט אשר יעבור את הבחינה בהצלחה ייכשל בקורס (כשהממוצע המשוקלל של המטלות והבחינה יהיה נמוך מ- 60).

כלל זה איננו חל על מטלות חובה או על מטלות שנקבע עבורן ציון מינימום.

מטלת מנחה (ממ"ן) 11

הקורס: שפות תכנות (20905)

חומר הלימוד למטלה: פרק 1 בספר הלימוד, פרק 3 במדריך הלמידה, חלקו השני של מדריך

הלמידה העוסק בשפת scheme ובסביבת העבודה racket וכן מדריכים באתר הקורס.

מספר השאלות: 3 משקל המטלה: 5 נקודות

סמסטר: ב2026 מועד הגשה: 29.3.2026

- שליחת המטלות תתבצע רק באמצעות מערכת המטלות המקוונת באתר הבית של הקורס.

שימו לב, בכל מקום במטלה בו נכתבה המילה סמל, הכוונה היא ל-scheme symbol.

אופן הגשת מטלה 11:

יש להגיש קובץ זיפ ובתוכו 2 קבצים: אחד קובץ טקסט המכיל בתוכו פתרונות לכל 3 השאלות

יחד עם דוגמאות הרצה לכל שאלה באמצעות שימוש ב-equal??, וקובץ שני הוא utils.scm

הנלווה לקוד של פרק 2 בספר (המספק לנו את equal??)

שאלה 1 (30 נקודות)

א) הפרוצדורה append בשפת Scheme מקבלת 2 רשימות ומאחדת אותם לרשימה אחת ע"י שרשור של הרשימה השניה לסוף של הראשונה. ממשו את append בעצמכם (כמובן אסור להשתמש ב-append של השפה). קראו לפרוצדורה בשם my_append להלן דוגמא:

```
>(my_append '(a b c) '(x y z))
```

```
(a b c x y z)
```

ב) ממשו כעת את append ע"י שימוש ב-foldr וללא רקורסיה ישירה (כלומר, במימוש לא תופיע קריאה ישירה לפרוצדורה אותה אתם נדרשים לממש). קראו לפרוצדורה בשם my_append_fr

שאלה 2 (35 נקודות)

כתבו פרוצדורה בשם filter המקבלת כפרמטרים: פרדיקט ורשימה. הפרוצדורה מחזירה רשימה חדשה המכילה רק את האיברים מהרשימה המקורית שעבורם ערכו של הפרדיקט היה אמת. הפרדיקט הוא פרוצדורה המקבלת ארגומנט יחיד ומחזירה #t או #f למשל, הפרדיקט number? מחזיר #t אם הארגומנט שלו הוא מספר, אחרת מחזיר #f. להלן דוגמא:

```
>(filter even? '(1 2 3 4 5 6))
```

```
(2 4 6)
```

ממשו את filter באמצעות שימוש ב-foldr, וללא רקורסיה ישירה (כלומר, במימוש לא תופיע קריאה ישירה לפרוצדורה אותה אתם נדרשים לממש)

שאלה 3 (35 נקודות)

כתבו פרוצדורה רקורסיבית בשם powerset המקבלת כפרמטר רשימה שטוחה (ללא רשימות מקוננות וללא חזרות של איברים באותה רשימה), הרשימה מייצגת קבוצה של איברים. הפרוצדורה תחזיר את קבוצת החזקה של הקבוצה שהתקבלה כפרמטר. התשובה המוחזרת תהיה רשימה של תתי רשימות, כאשר כל תת-רשימה מייצגת אחת מתת-הקבוצות בקבוצת החזקה. לדוגמא:

```
> (powerset '(3 2 1))  
((3 2 1) (3 2) (3 1) (3) (2 1) (2) (1) ( ))  
>(powerset '())  
(( ))
```

מטלת מנחה (ממ"ן) 12

הקורס: שפות תכנות (20905)

חומר הלימוד למטלה: פרק 2 בספר הלימוד, פרק 4 במדריך הלמידה.

מספר השאלות: 2 משקל המטלה: 5 נקודות

סמסטר: 2026 מועד הגשה: 19.4.2026

- שליחת המטלות תתבצע רק באמצעות מערכת המטלות המקוונת באתר הבית של הקורס.

לצורך פתרון מטלה זו עליכם ללמוד היטב את פרק 2 בספר הלימוד בליווי של פרק 4 במדריך הלמידה.

שאלה 1 (60 נקודות)

פולינום היא פונקציה מהצורה: $p(x) = a_0 + a_1x + a_2x^2 + \dots + a_kx^k$. דרגת הפולינום היא החזקה הגבוהה ביותר בעלת מקדם שונה מאפס.

poly הוא טיפוס נתונים מופשט (ADT) המייצג פולינום ותומך בפעולות הבאות:

- zero** – מחזירה פולינום ריק (ללא איברים), כלומר את $p(x)=0$.
- make-poly** – מקבלת כפרמטרים מקדם (coefficient) וחזקה (exponent) ומחזירה פולינום מתאים.
- למשל, **(make-poly 3 4)** מחזירה את הפולינום $p(x)=3x^4$.
- add-poly** – מקבלת זוג פולינומים ומחזירה פולינום המייצג את סכומם. למשל,

(add-poly (make-poly 3 4) (make-poly 2 1))

מחזירה את הפולינום $p(x) = 2x^1 + 3x^4$.

- degree** – מקבלת פולינום ומחזירה את דרגתו.
- למשל, **(degree (add-poly (make-poly 3 4) (make-poly 2 1)))** מחזירה את הערך 4.
- coeff** – מקבלת פולינום וחזקה ומחזירה את המקדם של האיבר בעל החזקה הנתונה. אם אין איבר כזה מוחזר 0.
- למשל, **(coeff (make-poly 3 4) 4)** מחזירה 3.
- אבל **(coeff (make-poly 3 4) 9)** מחזירה 0.
- is-zero?** – מקבלת פולינום ובודקת האם הוא הפולינום הריק. הפעולה מחזירה ערך בוליאני מתאים.
- למשל, **(is-zero? (zero))** מחזירה true. ואילו, **(is-zero? (make-poly 3 4))** מחזירה false.

- **print-poly** – מקבלת פולינום וממירה ומחזירה אותו בצורת רשימה המורכבת מזוגות סדורים, כאשר כל זוג מייצג מקדם וחזקה של גורם בפולינום.
- **Calc-poly** – מקבלת פולינום וערך מספרי, ומחשבת את ערכו של הפולינום עבור הערך הנתון.

(א) (8 נקודות)

כתבו אפיון לחתימות של הפעולות. סווגו את הפעולות השונות לסוגים (בנאים, מחלצים, מנבאים).

(ב) (16 נקודות)

כתבו אפיון אלגברי לכל אחת מהפעולות המגדיר את הסמנטיקה (המשמעות) שלהם.

(ג) (36 נקודות)

ממשו את **poly** ואת המנשק לעבודה איתו (הפעולות שהוגדרו למעלה). ע"י שימוש בכלי `define-datatype`

שאלה 2 (40 נקודות)

בעמוד 40 בספר הלימוד נתון מימוש לטיפול הנתונים **סביבה**. מימוש זה עושה שימוש **בייצוג פרוצדורלי לייצוג הסביבה**.

על סמך **מימוש זה**, כתבו פרוצדורה בשם `is-num?` המקבלת סביבה `e`, משתנה `x` ובודקת האם ל-`x` יש כריכה בסביבה `e` לערך מספרי. הפרוצדורה תחזיר ערך בוליאני מתאים. שימו לב, בשאלה זו הייצוג של סביבה הוא פרוצדורלי.

מטלת מנחה (ממ"ן) 13

הקורס: שפות תכנות (20905)

חומר הלימוד למטלה: פרק 3 בספר הלימוד, פרק 5 במדריך הלמידה, נספח B בספר הלימוד.

משקל המטלה: 5 נקודות

מספר השאלות: 2

מועד הגשה: 10.5.2026

סמסטר: 2026

- שליחת המטלות תתבצע רק באמצעות מערכת המטלות המקוונת באתר הבית של הקורס.

לצורך פתרון מטלה זו עליכם ללמוד היטב את פרק 3 בספר הלימוד בליווי של פרק 5 במדריך הלמידה, וכן בנספח B בספר הלימוד.

שאלה 1 (50 נקודות)

שאלה זו עוסקת בשדרוג של שפת "ויהי" (שפת LET) המתוארת בספר הלימוד בפרק 3 בעמודים 60-74.

בשאלה זו נוסיף לשפת "ויהי" יכולת של ביצוע המרה (casting) לביטוי. נרצה לתמוך הן בהמרה נרמזת והן בהמרה מפורשת. נוסיף לשפה ביטוי חדש המבצע casting בצורה מפורשת לערכו של ביטוי נתון. אופן ביצוע ההמרה יוסבר בהמשך.

להלן הדקדוק המגדיר את תחביר הביטוי החדש:

$Expression ::= \langle Type \rangle (Expression)$

`cast-exp (typ exp1)`

$Type ::= int$

`int-type ()`

$Type ::= bool$

`bool-type ()`

- ביטוי `cast-exp` מבצע המרה של ערכו של `exp1` לערך מהטיפוס המתואר ע"י `typ`, באופן הבא:
 - אם ערכו של `exp1` הוא מספרי ונדרשת המרה שלו לערך בוליאני, אזי יוחזר ערך בוליאני של `#t` אם ערכו המספרי של `exp1` שונה מאפס, אחרת יוחזר ערך בוליאני של `#f`.
 - אם ערכו של `exp1` הוא בוליאני ונדרשת המרה שלו לערך מספרי, אזי יוחזר ערך של 1 אם ערכו של `exp1` הוא `#t`, אחרת יוחזר ערך של 0.
 - אם נדרשת המרה של ביטוי לאותו טיפוס של הביטוי – אז לא תבוצע כל המרה ויוחזר ערכו של הביטוי.

בנוסף נרצה לשנות את המימוש של ביטוי if-exp כך שהוא יחושב עם המרה נרמזת של המפרש. כלומר, אם המתכנת ירשום ביטוי מספרי במקום ביטוי בוליאני בחלק התנאי של ביטוי if, במקום שתצא שגיאה, המפרש ידאג לבצע המרה נרמזת של הביטוי המספרי לביטוי בוליאני כפי שהוסבר קודם.

להלן מספר דוגמאות להמחשה:

דוגמה 1:

```
> (run " if 9 then 100 else 200")  
(num-val 100)  
>
```

בדוגמה זו, המתכנת רשם בחלק הבוליאני של ביטוי if ביטוי שאינו בוליאני, והמפרש ביצע המרה נרמזת בעצמו, 9 הומר לביטוי בוליאני של #t ולכן תוצאת התוכנית היא 100

דוגמה 2:

```
> (run " if 0 then 100 else 200")  
(num-val 200)  
> |
```

בדוגמה זו, המתכנת רשם בחלק הבוליאני של ביטוי if ביטוי שאינו בוליאני, והמפרש ביצע המרה נרמזת בעצמו, 0 הומר לביטוי בוליאני של #f ולכן תוצאת התוכנית היא 200

דוגמה 3:

```
> (run " -(100,<int>(zero?(0)))")  
(num-val 99)  
> |
```

בדוגמה זו, בוצעה פעולת חיסור, כאשר הביטוי השני של פעולת החיסור הוא המרה מפורשת של ביטוי בוליאני לביטוי מספרי, במקרה זה #t הומר ל-1, ולכן תוצאת התוכנית היא 99

המשך הדוגמאות בעמוד הבא

דוגמה 4:

```
> (run " if <bool> (9) then 100 else 200")  
(num-val 100)  
> |
```

בדוגמה זו, בוצעה המרה מפורשת של ביטוי חשבוני לביטוי בוליאני במסגרת ביטוי if.

דוגמה 5:

```
> (run "  
    let w = 40  
    in  
    if <bool>(w) then 500 else 700")  
(num-val 500)  
> |
```

גם בדוגמה זו בוצעה המרה מפורשת של ביטוי חשבוני לביטוי בוליאני במסגרת ביטוי if.

ממשו והטמיעו את השינויים הדרושים כפי שהוגדרו והודגמו בתוך שפת "ויהי" (שפת LET).

אופן הגשה:

הגשת קובץ זיפ במערכת המטלות המכיל את קבצי הקוד של המפרש של שפת "ויהי" כפי שנלווים לספר הלימוד (נמצאים באתר הקורס) ובתוכם הטמעת השינויים הדרושים. כמו כן, יש להוסיף דוגמאות הרצה תחת הקובץ tests.scm הנלווה למפרש השפה (את הטסטים הנוכחיים שעושים בעיות עקב הטמעת השינויים יש לשים תחת הערות) חובה להגיש את כל קבצי המפרש כפי שהם נלווים לספר הלימוד, ובתוכם השינויים הדרושים – לא יתקבלו פתרונות שלא בפורמט זה!

המשך בעמוד הבא

שאלה 2 (50 נקודות)

שאלה זו עוסקת בשדרוג של שפת "ויהי" (שפת LET) המתוארת בספר הלימוד בפרק 3 בעמודים 60-74.

בשאלה זו נרצה להוסיף לשפת "ויהי" ביטוי חדש מסוג לולאת do המוגדרת על פי הדקדוק הבא ובאופן המוסבר ומודגם לאחר מכן.

להלן הדקדוק של לולאת do :

$$\text{Expression} ::= \text{do} (\begin{array}{l} \{ < \text{Identifier Expression Expression} > \}^+ \\ \{ [\text{Expression Expression}] \}^+ \end{array})$$

<code>do-exp (ids inits steps bools results)</code>

הסבר לדקדוק :

ביטוי do-exp מגדיר כמות שרירותית (לפחות אחד) של משתנים מספריים (ids), למשתנים יש ערכים התחלתיים הנתונים ע"י ערכם של הביטויים inits, ובכל מחזור לולאה המשתנים גדלים בצעדים הנתונים ע"י ערכם של הביטויים steps. כמו כן, ללולאה כמות שרירותית (לפחות אחד) של ביטויים בוליאניים bools, בכל מחזור לולאה נבדקים הביטויים הבוליאניים, והראשון מביניהם שערכו אמת, יגרום ללולאה להיעצר ולהחזיר את ערכו המתאים של הביטוי מתוך results המתאים לביטוי הבוליאני שנמצא שערכו אמת. אם בבדיקת הביטויים הבוליאניים אין ביטוי שערכו אמת, הלולאה תעבור למחזור הבא שלה, כאשר כל אחד מהמשתנים גדל על פי הצעד המתאים לו, ואז שוב נבדקים הביטויים הבוליאניים כמו קודם. הלולאה תתבצע שוב ושוב, עד אשר ימצא מחזור לולאה שבו אחד התנאים הבוליאניים היה אמת. (כמובן שיכולה להתרחש לולאה אינסופית אך זו אחריותו של המתכנת ולא של המפרש)

הבהרות :

- אתחול של משתנה לולאה do יכול להכיר כבר את המשתנים שכבר אותחלו לפניו.
- קידום משתני לולאה נעשה על פי ערכם של המשתנים במצב הנוכחי

בעמוד הבא מופיעות מספר דוגמאות להמחשת אופן הפעולה של לולאת do

```
> (run "do (
      <a 0 4>
      <b 7 -(a,2)>
      <c -(b,1) -(a,b)>
      [zero? (b)          -(a,c)]
      [zero? (- (c,-2))   -(a,5)]
    )
")
(num-val 3)
>
```

בדוגמה זו, התוכנית מורכבת מלולאת do המגדירה 3 משתנים a,b,c. בתחילה a שווה לאפס, b שווה ל-7, c שווה ל-6 (b פחות 1). כעת נבדקים התנאים הבוליאנים לפי סדר הופעתם. התנאי הבוליאני הראשון בודק האם ערכו של b הוא 0, תנאי זה אינו מתקיים כרגע. התנאי הבוליאני השני בודק האם ערכו של c הוא -2, גם הוא לא מתקיים כרגע. מאחר ואף תנאי בוליאני לא התקיים, הלולאה ממשיכה למחזור הבא שלה, שבו היא תעבוד עם משתנים a,b,c שערכם הוא כעת:

a שווה ל-4, כי היה לו 0 והתווסף לו 4
b שווה ל-5, כי היה לו 7 והוא גדל ב-2, a-2, במצבו הנוכחי (לפני שקודם) היה 0 ולכן a-2 זה -2 ולכן b קטן ב-2 מ-7 ל-5
c שווה ל-1, כי היה לו 6 והתווסף לו a-b, ערכם של a ו-b (לפני שקודמו) היה 0 ו-7 בהתאמה ולכן c קטן ב-7 ונהיה -1

כעת נבדקים שוב התנאים הבוליאניים, וניתן לראות ששוב אף אחד מהם אינו נכון, ולכן הלולאה ממשיכה למחזור הבא שלה, עם משתנים a,b,c שערכם הוא:

a שווה ל-8, כי היה לו 4 והתווסף לו 4.
b שווה ל-7, כי היה לו 5, והוא גדל ב-2 שזה a-2 שזה 2
c שווה ל-2, כי היה לו -1 והוא גדל ב-a-b שזה -1

וכעת שוב נבדקים התנאים הבוליאניים (לפי הסדר בו הם כתובים) וכעת התנאי הבוליאני השני מתקיים! ולכן הלולאה נעצרת ומוחזר ערכו של הביטוי הכתוב לצד התנאי הבוליאני שערכו היה אמת, שבמקרה זה זה הביטוי a-5 שערכו עלפי ה-a הנוכחי הוא $8-5=3$ ולכן תוצאת התוכנית היא 3.

המשך דוגמאות נוספות בעמוד הבא

דוגמה 2:

```
> (run "do(
    [zero? (b)      -(a,c) ]
    [zero? (-(c,-2)) -(a,5) ]
  )
")
interp.scm:72:15: do-exp: do loop with no variables are not allowed
> |
```

בדוגמה זו יש לולאת do ללא הגדרת משתנים, על פי הדקדוק הנתון בשאלה נדרש לפחות משתנה אחד ולכן התקבלה הודעת שגיאה מתאימה.

דוגמה 3:

```
> (run "do(
    <a 0 4>
    <b 7 -(a,2)>
    <c -(b,1)  -(a,b)>
  )
")
interp.scm:74:19: do-exp: do loop with no booleans are not allowed
> |
```

בדוגמה זו יש לולאת do ללא הגדרת תנאים בוליאנים, על פי הדקדוק הנתון בשאלה נדרש לפחות תנאי בוליאני אחד ולכן התקבלה הודעת שגיאה מתאימה.

ממשו והטמיעו את השינויים הדרושים כפי שהוגדרו והודגמו בתוך שפת "ויהי" (שפת LET).

אופן הגשה:

הגשת קובץ זיפ במערכת המטלות המכיל את קבצי הקוד של המפרש של שפת "ויהי" כפי שנלווים לספר הלימוד (נמצאים באתר הקורס) ובתוכם הטמעת השינויים הדרושים. כמו כן, יש להוסיף דוגמאות הרצה תחת הקובץ tests.scm הנלווה למפרש השפה (את הטסטים הנוכחיים שעושים בעיות עקב הטמעת השינויים יש לשים תחת הערות) חובה להגיש את כל קבצי המפרש כפי שהם נלווים לספר הלימוד, ובתוכם השינויים הדרושים – לא יתקבלו פתרונות שלא בפורמט זה!

מטלת מנחה (ממ"ן) 14

הקורס: שפות תכנות (20905)

חומר הלימוד למטלה: פרק 3 בספר הלימוד, פרק 5 במדריך הלמידה, נספח B בספר הלימוד.

מספר השאלות: 3 משקל המטלה: 5 נקודות

סמסטר: 2026 מועד הגשה: 24.5.2026

- שליחת המטלות תתבצע רק באמצעות מערכת המטלות המקוונת באתר הבית של הקורס.

לצורך פתרון מטלה זו עליכם ללמוד היטב את פרק 3 בספר הלימוד בליווי של פרק 5 במדריך הלמידה, וכן בנספח B בספר הלימוד.

שאלה 1 (25 נקודות)

בשפת "וישגור" פרוצדורות מחושבות על פי כריכה לקסיקלית. דרך אחרת לחשב פרוצדורות היא ע"י כריכה דינמית (Dynamic binding). בכריכה דינמית, גוף הפרוצדורה מחושב באמצעות הרחבת הסביבה ממנה מתבצעת הקריאה לפרוצדורה. בניגוד לכריכה לקסיקלית שבה גוף הפרוצדורה מחושב על פי הסביבה בה מוגדרת הפרוצדורה. נסתכל למשל על התוכנית הבאה בשפת "וישגור":

```
let a=3
```

```
in let p = proc (x) -(x,a)
```

```
  a=5
```

```
in -(a, (p 2))
```

בכריכה דינמית, המשתנה a המופיע בגוף הפרוצדורה כרוך לערך 5 ולא לערך 3.

שימו לב, בדוגמא זו שולבה גם יכולת להגדיר מספר ביטויים בתוך ביטוי let.

(א) שנו את שפת "וישגור" כך שפרוצדורות יחושבו רק בכריכה דינמית. לשם כך השתמשו במימוש שבו פרוצדורות מיוצגות ע"י מבנה נתונים.

(ב) כיתבו באמצעות הפתרון של סעיף א', פרוצדורה רקורסיבית המקבלת פרמטר n שהוא מספר חיובי, ומחזירה את האיבר ה- n בסדרת פיבונצ'י $(1, 1, 2, 3, 5, 8, \dots)$

אופן הגשה: קובץ זיפ המכיל את קבצי המפרש המקוריים של שפת וישגור עם השינויים

הדרושים בהם. את הפתרון של סעיף ב' יש להוסיף כטסט בקובץ tests.sem ואת כל הטסטים

הקיימים לשים בהערות.

כל צורת הגשה אחרת לא תתקבל.

שאלה 2 (30 נקודות)

א) בשפת "וישגור" (שפת PROC) הוגדר שלפּרוצדורה יש רק ארגומנט יחיד. אך ניתן לעקוף מגבלה זו ולכתוב פרוצדורה עם מספר ארגומנטים ע"י שימוש בפרוצדורות שמחזירות בעצמן פרוצדורות. נסתכל למשל על התוכנית הבאה בשפת "וישגור" :

```
let f = proc (x) proc (y) ...  
in ((f 3) 4)
```

טכניקה זו נקראת Currying וכבר נתקלנו בה במסגרת השיעור הראשון בקורס. כתבו בשפת "וישגור" באמצעות שימוש בטכניקה זו, פרוצדורה בעלת 2 ארגומנטים המחזירה תשובה בוליאנית (כמובן בצורת exp-val) של true כאשר 2 המספרים נגדיים אחד לשני. אחרת יוחזר false. למשל, עבור המספרים 4 ו-4 יוחזר true.

יש לפתור רק על ידי שימוש בדקדוק הקיים של שפת PROC (לא ניתן להרחיב את השפה במרכיבים חדשים)

ב) להלן תוכנית בשפת "וישגור".

```
let makemult = proc (maker)  
  proc (x)  
    if zero? (x)  
    then 0  
    else -(((maker maker) -(x,1)), -4)  
in let times4 = proc (x) ((makemult makemult) x)  
in (times4 3)
```

מהו הערך של תוכנית זו?

ג) כתבו בשפת "וישגור" פרוצדורה בשם f המקבלת מספר שלם חיובי n, מחשבת ומחזירה את סכום המספרים מ-1 עד n. לשם כך השתמשו בטכניקה מסעיף ב', וכן ב-Currying.

אופן הגשת שאלה 2:

יש להגיש קובץ זיפ עם קבצי המפרש של שפת "וישגור" כפי שמצורפים לספר הלימוד (ולא בשום פורמט אחר), את הפתרונות שלכם לסעיפים א+ב+ג יש לשלוח כטקסטים בקובץ tests.scn

שאלה 3 (45 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81.

בשאלה זו נוסיף לשפת "וישגור" יכולת בסיסית של Reflection. Reflection היא היכולת של תוכנית לתחקר בזמן ריצה מרכיבים של התוכנית עצמה לקבלת מידע עליהם וביצוע פעולות עליהם. בשאלה זו נרצה לממש יכולת לתיחקור של משתנה בזמן ריצה וקבלת מידע על טיפוסו.

נוסיף לשפה מספר ביטויים חדשים.

להלן הדקדוקים של הביטויים החדשים:

Expression ::= get-type (identifier)

get-type-exp (id)

Expression ::= isBool? (Expression)

isBool?-exp (ex)

Expression ::= isNum? (Expression)

isNum?-exp (ex)

Expression ::= isProc? (Expression)

isProc?-exp (ex)

- ביטוי get-type-exp מקבל id המציין שם של משתנה ומחזיר expval המתאר את טיפוסו (שימו לב, זה ביטוי עם סוג חדש של תשובה מוחזרת) הביטוי יבדוק מה מייצג המשתנה (מספר, בוליאני, פרוצדורה) ויחזיר ייצוג של טיפוס מתאים.
- הביטויים isBool?-exp , isNum?-exp , isProc?-exp מקבלים ביטוי ex, שאמור להיות מסוג get-type-exp ובודקים האם תוצאתו (שהיא תשובה המייצגת טיפוס) היא בוליאנית, מספרית או פרוצדורה בהתאמה. הביטויים מחזירים תשובה בוליאנית (bool- val) מתאימה.

המשך בעמוד הבא

להלן דוגמה להמחשת היכולות החדשות של reflection :

דוגמה:

```
> (run "let a= 10
    in
      let p= proc (w) -(w,a)
      in
        let q= proc (something)
          let t= get-type(something)
          in
            if isNum?(t)
            then
              (p something)
            else
              if isBool?(t)
              then
                if something then 500 else 800
              else
                (something 50)
          in
            (q zero?(a))")
(num-val 800)
>
```

בדוגמה זו מוגדרת פרוצדורה בשם q המקבלת פרמטר כלשהו בשם something, בגוף הפרוצדורה מתבצע תיחקור בזמן ריצה לגבי טיפוסו של something באמצעות היכולות החדשות שהתווספו לשפה. t מכיל תוצאה המתארת את טיפוסו של something, ולאחר מכן נבדק הטיפוס t מול מגוון טיפוסים שונים, ולפיהם הפרוצדורה פועלת בהתאם. בהמשך מופעלת הפרוצדורה q עם פרמטר בוליאני שערכו #f, מה שגורם במהלך ביצועה של q ותיחקור הפרמטר שהגיע אליה לכך שיוחזר 800

ממשו והטמיעו את השינויים הדרושים כדי שהוגדרו והודגמו בתוך שפת "וישגור" (שפת PROC).

אופן הגשה:

הגשת קובץ זיפ במערכת המטלות המכיל את קבצי הקוד של המפרש של שפת "וישגור" כדי שנלווים לספר הלימוד (נמצאים באתר הקורס) ובתוכם הטמעת השינויים הדרושים. כמו כן, יש להוסיף דוגמאות הרצה תחת הקובץ tests.scm הנלווה למפרש השפה (את הטסטים הנוכחיים שעושים בעיות עקב הטמעת השינויים יש לשים תחת הערות) חובה להגיש את כל קבצי המפרש כדי שהם נלווים לספר הלימוד, ובתוכם השינויים הדרושים – לא יתקבלו פתרונות שלא בפורמט זה!

מטלת מנחה (ממ"ן) 15

הקורס: שפות תכנות (20905)

חומר הלימוד למטלה: פרק 4 בספר הלימוד, פרק 6 במדריך הלמידה.

משקל המטלה: 5 נקודות

מספר השאלות: 1

מועד הגשה: 7.6.2026

סמסטר: ב2026

- שליחת המטלות תתבצע רק באמצעות מערכת המטלות המקוונת באתר הבית של הקורס.

לצורך פתרון מטלה זו עליכם ללמוד היטב את פרק 4 בספר הלימוד בליווי של פרק 6 במדריך הלמידה.

שאלה 1 (100 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "וירמוז" (IMPLICIT-REFS). בשאלה זו נשנה את האופן בו פרוצדורות מוגדרות ומופעלות, ונאפשר למתכנת לבצע העמסה (overloading) לפרוצדורה שכבר הוגדרה, כך שבהפעלת הפרוצדורה תופעל הגרסה המתאימה שלה במידה וקיימת, בהתאם לסוג הפרמטר הנשלח. לשם כך יש צורך בהוספת ביטוי חדש לביצוע ההעמסה, שינוי ביטוי `proc-exp` הקיים בשפה להגדרת פרוצדורה, ועדכון תהליך ההפעלה של פרוצדורות.

להלן הדקדוקים המגדירים את ביטוי `proc-exp` המעודכן, וכן את הביטוי החדש להעמסה:

Expression ::= proc (type identifier) Expression

`proc-exp (typ var body)`

type ::= int

`int-type ()`

type ::= bool

`bool-type ()`

type ::= func

`func-type ()`

Expression ::= overload identifier with (type identifier) Expression

`overload-exp (p-name typ var body)`

המשך בעמוד הבא

הסבר הדקדוקים וההתנהגותם:

- ביטוי proc-exp להגדרת פרוצדורה עודכן והוא מכיל גם טיפוס לשם הפרמטר של הפרוצדורה כחלק מהגדרתה.
- ביטוי overload-exp החדש מקבל שם של פרוצדורה שכבר הוגדרה ומאפשר לשמור גרסה נוספת וחדשה שלה, הגרסה הנוספת החדשה מקבלת פרמטר var מטיפוס typ ומבצעת את מה ש-body מייצג. ניתן להעמיס פרוצדורה מספר פעמים ללא הגבלה.
 - אם ביטוי overload-exp מנסה לבצע העמסה של משתנה שאינו מייצג פרוצדורה כלל – תיזרק שגיאה מתאימה על כך.
 - אם ביטוי overload-exp מנסה לבצע העמסה של משתנה שלא קיים כלל – תיזרק שגיאה.
 - אם ביטוי overload-exp מבצע העמסה לפרוצדורה קיימת עם גרסה נוספת בעלת פרמטר מסוג שונה, הגרסה הנוספת תישמר בנוסף לגרסה הקיימת, וכאשר תהיה דרישה להפעיל את הפרוצדורה עם פרמטר מסוג מסוים תופעל הגרסה המתאימה של הפרוצדורה, במידה וקיימת עבור סוג הפרמטר, אחרת תוחזר שגיאה מתאימה.
 - אם ביטוי overload-exp מבצע העמסה לפרוצדורה קיימת עם גרסה נוספת בעלת פרמטר מסוג שכבר קיימת עבורו גרסה קודמת, אזי הפרוצדורה תועמס עם הגרסה החדשה, והגרסה הקודמת לא תהיה רלוונטית יותר.

להלן מספר דוגמאות להמחשת השימוש וההתנהגות של היכולות החדשות המבוקשות:

דוגמה 1:

```
> (run "let sum = 0
      in
        begin
          overload sum with (int b) -(b,50);
          (sum 100)
        end")
❌❌ interp.scm:94:23: overload-exp: cannot overload non procedure sum
```

בדוגמה זו יש ניסיון להעמיס את sum שאינו מייצג כלל פרוצדורה, ועל כן התקבלה שגיאה.

דוגמה 2:

```
> (run "let sum = 0
      in
        begin
          overload q with (int b) -(b,50);
          (q 100)
        end")
❌❌ environments.scm:30:10: apply-env: No binding for q
>
```

בדוגמה זו יש ניסיון להעמיס משתנה שלא קיים כלל, ולכן התקבל שגיאה.

המשך הדוגמאות בעמוד הבא

```
> (run "let sum = 0
in
  let p= proc (int a) -(a,10)
  in
    begin
      set sum = -(sum, -(0, (p 100)));
      overload p with (bool b) if b then 700 else 1000;
      set sum = -(sum, -(0, (p zero?(5))));
      overload p with (int r) -(r,300);
      set sum = -(sum, -(0, (p 100)));
      overload p with (func f) (f 30);
      set sum = -(sum, -(0, (p proc (int w) -(w,50))));
      sum
    end")
(num-val 870)
```

בדוגמה זו, מוגדר משתנה sum עם ערך התחלתי של 0. וכן פרוצדורה בשם p שכרגע יודעת לקבל פרמטר int ולהחזיר אותו מופחת ב-10. לאחר מכן מבוצעות מספר העמסות לפרוצדורה p וכן עדכון של משתנה sum.

בתחילה ערכו של sum מתעדכן, ומתווספת לו התוצאה המוחזרת מהפעלת p על 100, מאחר ויש ל-p גרסה היודעת לקבל פרמטר int, הופעלה גרסה זו שהחזירה 90 (הפחיתה 10 מ-100). ולכן ערכו הנוכחי של sum הוא 90. לאחר מכן, הפרוצדורה הועמסה עם גרסה היודעת לקבל פרמטר בוליאני, ושוב עודכן ערכו של sum והתווספה לו תוצאת קריאה ל-p עם פרמטר בוליאני, שהחזירה 1000, ולכן ערכו של sum כעת הוא 1090. לאחר מכן, p הועמסה שוב עם גרסה היודעת לקבל פרמטר int, ומאחר וכבר ישנה גרסה ישנה כזו, הגרסה הישנה לא תקפה יותר. כעת שוב מתבצעת קריאה ל-p עם פרמטר מספרי של 100, והפעם מופעלת הגרסה העדכנית עם פרמטר מספרי, המחזירה 200- המתווסף ל-sum, ולכן ערכו של sum כרגע הוא 890. לבסוף p מועמסת עם גרסה היודעת לקבל פרמטר מסוג פרוצדורה, ולאחר מכן, גרסה זו מופעלת ומחזירה במקרה זה, 20-, המתווסף לערכו של sum, ולכן מצבו הנוכחי של sum הוא 870. והוא מוחזר כתשובה סופית של כל התוכנית.

המשך הדוגמאות בעמוד הבא

דוגמה 4:

```
> (run "let sum = 0
    in
    let p= proc (int a)  -(a,10)
    in
    (p zero?(5))")
interp.scm:145:10: apply-procedure: No procedure found with #(struct:bool-type) argument
>
```

בדוגמה זו ישנו ניסיון להפעיל פרוצדורה p שאין עבורה גרסה עם פרמטר בוליאני, ולכן התקבלה שגיאה מתאימה.

ממשו והטמיעו את השינויים הדרושים בתוך שפת "וירמוז" (שפת IMPLICIT-REFS).

אופן הגשה:

הגשת קובץ זיפ במערכת המטלות המכיל את קבצי הקוד של המפרש של שפת "וירמוז" כפי שנלווים לספר הלימוד (נמצאים באתר הקורס) ובתוכם הטמעת השינויים הדרושים. כמו כן, יש להוסיף דוגמאות הרצה תחת הקובץ tests.scm הנלווה למפרש השפה (את הטסטים הנוכחיים שעושים בעיות עקב הטמעת השינויים יש לשים תחת הערות) חובה להגיש את כל קבצי המפרש כפי שהם נלווים לספר הלימוד, ובתוכם השינויים הדרושים – לא יתקבלו פתרונות שלא בפורמט זה!

מטלת מנחה (ממ"ן) 16

הקורס: שפות תכנות (20905)

חומר הלימוד למטלה: פרק 7 בספר הלימוד, פרק 6 במדריך הלמידה.

מספר השאלות: 1 משקל המטלה: 5 נקודות

סמסטר: ב2026 מועד הגשה: 14.6.2026

- שליחת המטלות תתבצע רק באמצעות מערכת המטלות המקוונת באתר הבית של הקורס.

לצורך פתרון מטלה זו עליכם ללמוד היטב את פרק 4 בספר הלימוד בליווי של פרק 6 במדריך הלמידה.

שאלה 1 (100 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
let p = proc (a: ?) (a 6)
in
(p proc (b: int) zero? (b))
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים). לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב-substitution ו-unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.